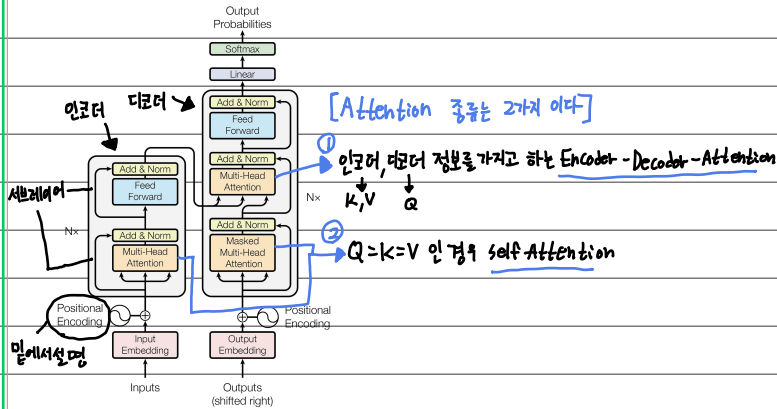


Transformer [논문: Attention is All you need]

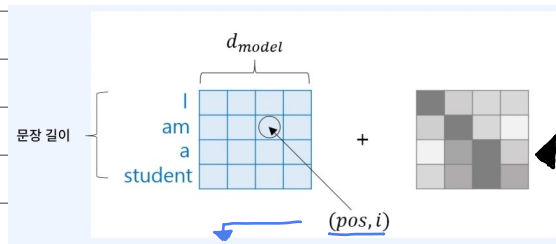
- 이전의 논문에서 RNN 구조 + Attention 구조를 통해 Context vector에 정보가 손실되는 부분을 해결했다. ▶ RNN 있을 때는 순차적으로 해야했다.
- 그래서 Transformer 저자는 Attention 구조만 사용하면 병렬처리와 성능향상을 할 수 있다고 생각했다.

아키텍처



Positional Encoding

- 인풋 데이터가 처음 들어오면 제일 먼저 위치 정보를 더해준다.
- 기존의 RNN은 데이터가 순차적으로 들어감으로써 위치정보가 자연스럽게 추가되었다.
- 하지만 Transformer는 데이터가 병렬로 한번에 들어감으로써 누가 몇번째 데이터인지 모른다.
- 위의 이유 때문에 아래 이미지와 같이 Positional Encoding 행렬을 만들어 추가한다.



i = 임베딩 벡터의 차원 $[d]$

- ▶ 논문에서는 차원 d 은 512로 고정했다. ▶ 모든 input의 차원
- ▶ 인코더, 디코더의 모든 임베딩 차원을 512로 고정했다

pos = 입력문장에서의 임베딩 벡터의 위치 $[I, am, a, student]$

$$PE_{(pos, 2i)} = \sin(pos/10000^{2i/d_{model}})$$

$$PE_{(pos, 2i+1)} = \cos(pos/10000^{2i/d_{model}})$$

→ 이것을 가지고 행렬을 만들어 위치 행렬을 만든다.

▶ 왜 이런 복잡한 수식 사용 하는 거?

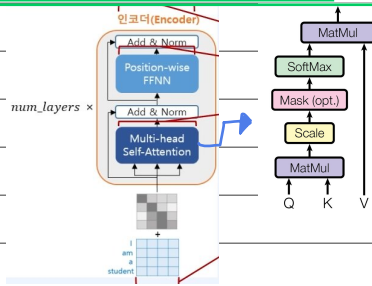
▶ 많은 어휘 중 1개는 일반화 때문이다.

만약 수식을 사용하면 이 2, 3 단어만 학습 데이터보다

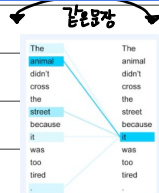
긴 데이터가 들어오면 성능 저하

다른 어휘는 $\sin, \cos$ 이니까 모델이 3만어, 2만어 뒤를 아껴가능

Self-Attention 과 Multi-Head Attention

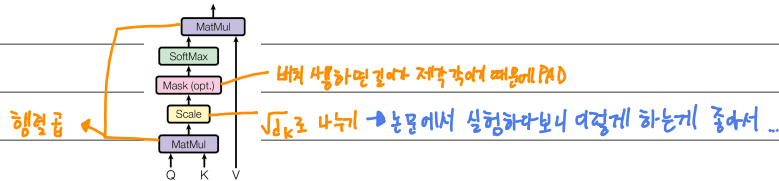


하는 이유

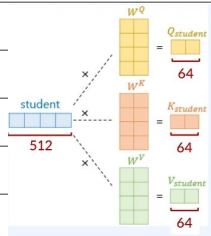


- 만약 문장을 전체를 보지 않고 (i)만 본다면 (ii)이 의미하는게 무엇인지 모르는 상황이 발생
- 그래서 (ii)가 무엇을 의미하는지 문장 전체를 봐야한다.
- 단어 하나가 자기 외 문장 전체를 보는 것이 self attention 이다.

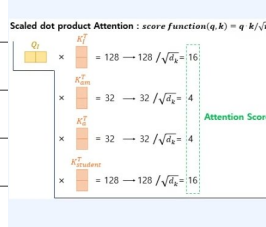
과정(예시) 1번 → Self Attention



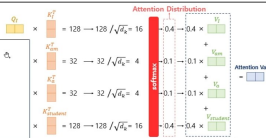
• I am a student



- 위에 행렬에서 각 단어에서 Q, K, V 벡터 값을 가져오기 위해 W^Q, W^K, W^V 를 행렬 곱을 통해 64차원 Q, K, V 값을 가져온다.
- 각 문장마다 W^Q, W^K, W^V 가 동일하다
- 만약 각 문장마다 다르다면 문장이 짧고 길때 문제가 생긴다.

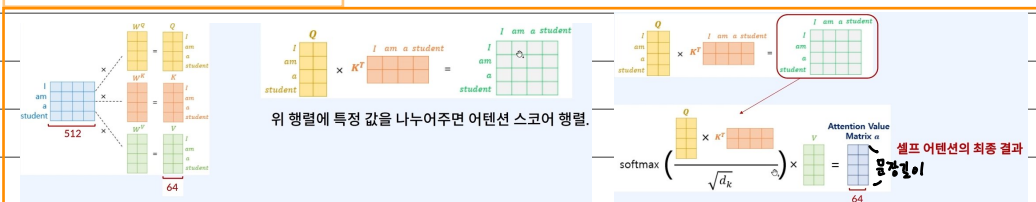


- 이후 'I'의 Attention score를 가져오기 위해서는 'I'의 Q와 각 단어의 K를 내적을 해준다.
- 이후 $\sqrt{d_k}$ 로 나눠준다.



- 결과값을 SoftMax를 해준다
- 그결과를 마저 막 각 단어의 V와 곱한뒤 모두 더해 'I'의 Attention Value를 뽑아낸다.

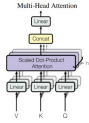
위에 과정을 한큐로 나타내면 아래와 같다.



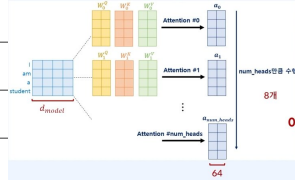
- 셀프어텐션의 모든 차원은 $d_k=512$ 차원인데 이상태라면 다음 과정이 막힌다.
- 그래서 아래와 정을 통해 512차원을 맞춰준다.

과정(예제) 2번 - Multi-Head Attention

- 위에서 64 차원을 512로 만들기 위해 위의 과정을 8번진행한다.
- 처음부터 64가 아닌 512차원을 하지 않은 이유는 관점을 8개로 만들어서 정보로 풍부하게 얻기 위함이다.
- 그래서 이름이 Multi-Head Attention 인 것이다.

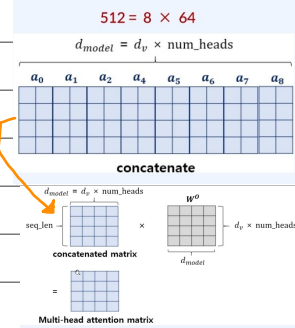


Multi-Head Attention



- 이렇게 총 8번진행한다.
- 당연히 다른 관점을 얻기 위해서 각 관점마다 W^Q, W^K, W^V 는 관점마다 다르다.

- 위에 결과를 단순히 Concat 으로 진행한다.



- 이후 정의한 계명의 W^O 를 곱해줌으로써 Multi-head attention matrix 가 나오게 된다.

1번, 2번 과정을 수식으로

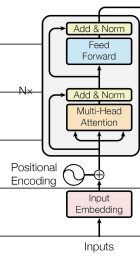
- 설명은 비교적 진행하지 않는다.

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O$$

where $\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$

Position-wise Feed Forward Neural Network



- 위에서 Multi-Head Attention 까지 끝나고
- Add & Norm** 부분만 거치는데 단순히 정보손실을 줄이기 위해 입력값을 출력에 더하고 Norm 사용다. (Residual 쪽처럼)
- 그리고 단순한 NN를 거치게 끝이다.

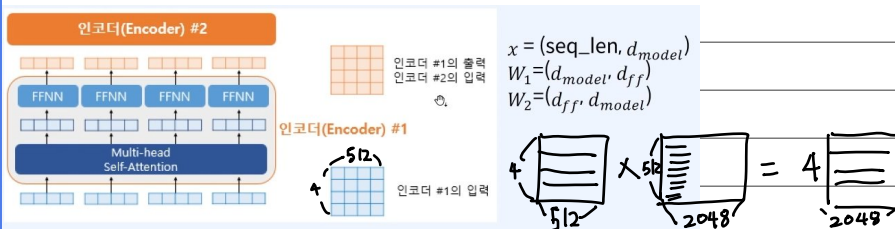
세부과정

- 위에서 말한것처럼 단순한 NN이다.

$$F_1 = xW_1 + b_1$$

$$\text{활성화 함수: ReLU} \\ F_2 = \max(0, F_1)$$

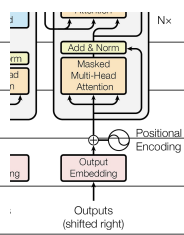
$$F_3 = F_2W_2 + b_2$$



- 위에서 문장길이 (4) $\times$ d_{model} (512) 을 가져와서 FFNN을 진행한다.
- 논문에서는 $d_{\text{model}} = 512$, $d_{\text{ff}} = 2048$ 을 사용해 결과가 동일하게 문장길이 (4) $\times$ 512 가 결과로 나온다.
- 이후 동일하게 Add & Norm 진행한다.
- 이거를 몇번 진행한다.

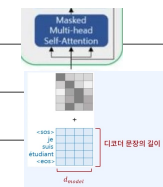
• 다거 이해 안가면 외적, 내적 다시 하라

Masked Multi-Head Attention → Decoder 부분



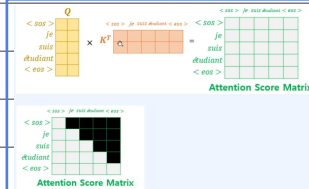
- 이전의 Multi-Head-Attention과 비슷하지만 Masked 가 들어간다. (밑에서 설명)
- 순서대로 설명 시작한다.

과정 ① - 디코더의 들어갈 매터 준비



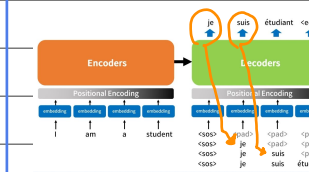
- 인코더처럼 Position Encoding을 적용시켜서 들어간다.
- 문장을 다 넣어버리면 '충분' 할때는 어떻게해? 라고생각하지만 다음의 Masked 작업이 해결해준다.
- 우선 준비해서 넣어준다.

과정 ② - Masked Multi-Head Attention



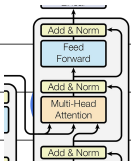
- 기존에 Multi-Head Attention 과정중에 Soft max 작전에 $- \infty$ 로 Mask 작업을한다.
- Mask작업을하면 모델은 현재시점에서 다음단어를 고려하지 못하고 현재와 과거 정보만으로 Attention-Score를 만들어낸다.

- 이렇게하면 학습할때 현재시점 score와 미래 score가 영향을 서로줘서 모델이 이상하게 판단하지 않을까? → 아니, 위에다해보면 각단어가 score 땀다 서로 영향을 주는 것은 soft max 뿐이고 그래서 Mask로 지워버릴거지 완전 독립의 문제이다.



- 나중에 infer 할때는 하나씩 들어가고 출력과 입력을 합쳐 다시 input으로 들어가서 문제가 없다.
- 이때에는 Add와 Norm작업을 똑같이 진행한다.

Encoder-Decoder-Attention



- 여기는 Multi-Head Attention 이라고하지만 K,V,Q가 좀달라서 Encoder-Decoder-Attention 이라고 부른다.
- Q = Decoder 벡터
- K,V = Encoder 벡터가 들어가서 Multi Head Attention 작업을 한다.

